

Comdove Mock Server — API Tech Spec

2026-09-19 · @Someone

Companion to [Doc](#) . Defines every API the mock exposes, side by side with the official WhatsApp Cloud API it emulates, plus the auth model, webhooks, WebSocket protocol and environment variables.

1. Scope and authentication model

The mock exposes three API families, each with a different auth rule. The rule in one line: **auth exists only where Meta has it, and there it is a fake bearer token from an environment variable; everything mock-specific has no auth at all.** The server runs on localhost or the office LAN only and is never exposed publicly.

API family	Consumed by	Auth
Emulated Meta Cloud API (/v23.0/{phone_number_id}/messages)	Comdove wat-backend	Bearer token, same header as Meta: Authorization: Bearer <token>. Token is fake, generated at number registration, read by Comdove from its existing env variable. Wrong or missing token returns Meta's 401 error JSON
Control API (/api/...)	Admin UI, client grid, future test automation	None
WebSocket (/ws)	Client grid tiles	None; the group name in the connect message claims the session lock

Why keep the bearer token at all: Comdove already sends it on every Graph API call. If the mock ignored auth, the token code path in wat-backend would go untested and a broken token header would pass tests and fail in production. The token check costs one string

compare and keeps the mock honest.

Webhooks in the reverse direction keep Meta's authentication too: every webhook the mock posts to Comdove is signed with X-Hub-Signature-256 using the shared app secret from env, so Comdove's signature verification stays switched on during tests.

2. API surface map: official vs mock

The mock implements the smallest slice of the [Cloud API](#) that Comdove's text path touches. Everything else fails loudly, never silently.

Official Cloud API area	Official endpoint	Mock v1
Send message (text)	POST /v23.0/{phone_number_id}/messages	Emulated in full for type text
Mark message as read	Same endpoint, body {status: "read", message_id}	Emulated (Should): accepting it triggers the read status webhook
Inbound message webhooks	Meta posts to the app's webhook URL	Emulated: mock posts to Comdove's webhook URL
Status webhooks (sent/delivered/read)	Same webhook URL	Emulated
Webhook verify handshake	GET with hub.mode, hub.verify_token, hub.challenge	Emulated: mock performs the GET once when its webhook URL is configured, so Comdove's verify endpoint is also tested
Media upload/download	/v23.0/{phone_number_id}/media	Not implemented
Template management	/v23.0/{waba_id}/message_templates	Not implemented

Official Cloud API area	Official endpoint	Mock v1
Phone number management	<code>/waba_id/phone_numbers</code>	Not implemented; registration happens on the mock's own control API

Rule for everything not implemented: any other path or message type returns HTTP 400 with a Meta-shaped error object (code 100), message text stating "not implemented in comdove-mock". Comdove code that strays outside the supported slice fails immediately with a readable reason instead of hanging or half-working.

Versioning: the mock accepts any `/vXX.X/` prefix and ignores it, since Comdove pins its own Graph version in config. Payload shapes follow the v23.0 reference.

3. Send Message API (emulated)

Request, identical to [Meta's](#):

```
POST /v23.0/{phone_number_id}/messages
Authorization: Bearer {token}
Content-Type: application/json

{
  "messaging_product": "whatsapp",
  "recipient_type": "individual",
  "to": "919876543210",
  "type": "text",
  "text": { "preview_url": false, "body": "Hello from Comdove" }
}
```

Success response, 200:

```
{
  "messaging_product": "whatsapp",
  "contacts": [ { "input": "919876543210", "wa_id": "919876543210" } ],
  "messages": [ { "id": "wamid.MOCK-a1b2c3d4e5f6" } ]
}
```

Validation, in order: bearer token matches the registered token for {phone_number_id} (else 401), messaging_product equals "whatsapp" and type equals "text" (else 400 code 100), "to" is a registered customer number in some group (else the undeliverable error). On success the message is stored, routed to the recipient's tile or queue, and the sent status webhook fires.

wamid format: "wamid MOCK-" plus a unique suffix. Same field, same place as Meta's id; the MOCK prefix makes test data unmistakable in Comdove's database.

Mark as read (Should): POST to the same endpoint with {"messaging_product": "whatsapp", "status": "read", "message_id": "wamid..." } returns {"success": true} and marks the inbound message read in the tile.

4. Error responses

All errors use Meta's envelope so wat-backend's error handling runs unchanged:

```
{
  "error": {
    "message": "(#131026) Message undeliverable",
    "type": "OAuthException",
    "code": 131026,
    "error_data": { "messaging_product": "whatsapp", "details": "Recipient is not a registered mock number" },
    "fbtrace_id": "MOCK-trace-000123"
  }
}
```

Trigger	HTTP	code	Meaning
Missing or wrong bearer token	401	190	Invalid OAuth access token
Malformed body, wrong messaging_product, unsupported type or path	400	100	Invalid parameter / unsupported request
"to" not a registered customer number	400	131026	Message undeliverable
Forced rate limit (see below)	400	130429	Rate limit hit

Deterministic error injection. Real Meta errors are hard to trigger on demand, so the mock adds one testing hook: a request header `X-Mock-Force-Error: {code}` makes the emulator return exactly that error for that request. This lets a tester (or later, a Playwright test) exercise Comdove's retry and error-surfacing paths on cue. The header is mock-only and is listed as a divergence in section 9.

5. Webhooks to Comdove

Shapes follow the [messages webhook reference](#). Both payload kinds share the entry/changes/value envelope with field "messages"; a messages array means an inbound message, a statuses array means a status update.

Inbound (tester typed in a tile):

```
{
  "object": "whatsapp_business_account",
  "entry": [ {
    "id": "MOCK-WABA-1",
    "changes": [ {
      "field": "messages",
      "value": {
        "messaging_product": "whatsapp",
        "metadata": { "display_phone_number": "918888800001",
"phone_number_id": "MOCK-PN-1" },
        "contacts": [ { "profile": { "name": "Tile 919876543210" }, "wa_id":
"919876543210" } ],
        "messages": [ { "from": "919876543210", "id": "wamid.MOCK-...",
"timestamp": "1758270000", "type": "text", "text": { "body": "how much?" } } }
      ]
    } ]
  } ]
}
```

Status (one webhook per transition; sent fires on API accept, delivered on tile/group receipt, read when the tester opens the chat):

```
{
  "object": "whatsapp_business_account",
  "entry": [ {
    "id": "MOCK-WABA-1",
    "changes": [ {
      "field": "messages",
      "value": {
        "messaging_product": "whatsapp",
        "metadata": { "display_phone_number": "918888800001",
"phone_number_id": "MOCK-PN-1" },
        "statuses": [ { "id": "wamid.MOCK-...", "status": "delivered",
"timestamp": "1758270031", "recipient_id": "919876543210" } ]
      }
    } ]
  } ]
}
```

Signature. Every POST carries X-Hub-Signature-256: sha256={HMAC-SHA256 of the raw request body, keyed with APP_SECRET}, byte-identical to Meta's scheme, so Comdove's verification stays on.

Verify handshake. When the mock starts (or the webhook URL changes), it performs Meta's subscription GET against Comdove's webhook endpoint with hub.mode=subscribe, hub.verify_token={WEBHOOK_VERIFY_TOKEN} and a random hub.challenge, and expects the challenge echoed back with 200.

Delivery and retries. Non-200 or a 5-second timeout triggers up to 3 retries with short backoff (1s, 5s, 15s). Every attempt and outcome shows in the admin log. Read behavior mirrors Meta's optimization: if a message is read before its delivered webhook was sent (a group reconnect straight into an open chat), the mock may send read without a preceding delivered.

6. Control API (mock-only, no auth)

Everything the admin UI does exists first as a plain HTTP endpoint under /api. No auth headers anywhere. The client grid and admin UI call these; phase 2 test automation will call the same ones.

Method + path	Body	Returns	Purpose
POST /api/business-numbers	{display_number, label}	{phone_number_id, token}	Register a business number (FR-01)
GET /api/business-numbers	-	list	Inventory for admin UI
POST /api/groups	{name, numbers: [up to 10]}	group	Create a client group (FR-15); numbers auto-register as customers
GET /api/groups	-	list with free/locked status	Launch page and lock display (FR-09, FR-16)
POST /api/presence	{number, online}	200	Set a tile online/offline without the UI
POST /api/inject	{from, to, body}	wamid	Fire an inbound message as if typed in a tile; fires the webhook to Comdove
GET /api/log? limit=100	-	message list with status timelines and webhook outcomes	Admin log (FR-11)
POST /api/reset	{keep_numbers: true}	200	Wipe messages and queues; numbers and groups kept when asked (FR-12)

Design notes: /api/inject is the automation twin of typing in a tile; with it and /api/presence, an end-to-end test can run without a browser at all, keeping the browser only for what must be visually verified. All responses are plain JSON; errors are {error: {message}} with 4xx, no Meta envelope here since this side of the mock is not pretending to be Meta.

7. WebSocket protocol (client grid <-> mock server)

One WebSocket connection per group at `ws://{host}/ws`. JSON messages, each `{type, ...payload}`. No auth; the claim message takes the group's session lock, and the lock releases when the socket closes.

Client to server

type	Payload	Effect
group.claim	{group}	Claim the group. Server replies group.claimed with a full state snapshot (tiles, histories, queued messages), or group.locked if another session holds it (FR-16, FR-17)
message.send	{from, to, body}	Tester sent a text from a tile; server stores it and fires the inbound webhook to Comdove (FR-07)
tile.presence	{number, online}	Toggle a tile; going online flushes that tile's queue (FR-05)
chat.read	{number, peer}	Tester opened a chat; server fires read status webhooks for unread messages in it (FR-06)

Server to client

type	Payload	Effect
group.claimed	{group, tiles: [{number, online, history, queued}]}	Snapshot on successful claim; the tile grid renders from this alone
group.locked	{group, since}	Claim refused; client returns to the launch page
message.new	{to, message}	New message for an online tile, live delivery (FR-04)
queue.flush	{number, messages: []}	Queued messages delivered in order after a tile or group comes back (FR-05, FR-17)
message.status	{wamid, status}	Mirror of status changes so tiles can show ticks

Heartbeat: standard WebSocket ping every 15s; a missed pong closes the socket and releases the lock, so a killed browser never wedges a group (FR-16).

8. Environment variables

All secrets and keys live in env on both sides; nothing is hardcoded and nothing else is configured.

Mock server (.env)

Variable	Example	Purpose
PORT	4020	HTTP + WebSocket port
COMDOVE_WEBHOOK_URL	http://localhost:3000/webhooks/whatsapp	Where inbound and status webhooks go
APP_SECRET	mock-app-secret-1	HMAC key for X-Hub-Signature-256; must equal Comdove's app secret setting
WEBHOOK_VERIFY_TOKEN	mock-verify-1	Sent as hub.verify_token in the handshake; must equal Comdove's verify token setting
DB_PATH	./mock.sqlite	SQLite file; delete it for a factory reset

Comdove wat-backend (.env, existing variables, values changed for testing)

Variable	Test value	Note
GRAPH_API_BASE_URL (or equivalent)	http://localhost:4020	The single switch between Meta and the mock. If wat-backend hardcodes graph.facebook.com today, extracting this variable is a prerequisite task
WhatsApp access token per number	token returned by POST /api/business-numbers	Same variable Comdove already uses; only the value is fake

Variable	Test value	Note
App secret	mock-app-secret-1	Keeps signature verification on
Verify token	mock-verify-1	Keeps the handshake path on

9. Known divergences from the official API

Documented so nobody mistakes a mock behavior for a Meta behavior.

Area	Meta	Mock
Auth	Real OAuth system-user tokens with scopes and expiry	Static fake token per number, string compare only
Control plane	Admin/control endpoints do not exist; numbers come from WABA onboarding	/api/* endpoints, no auth, local network only
wamid	Opaque base64-like id	"wamid.MOCK-" prefix, deliberately recognizable
Status webhook extras	conversation and pricing objects on some statuses (omitted from v24 onward)	Omitted entirely
Webhook retries	Increasing backoff up to roughly 24 hours	3 attempts, 1s/5s/15s
24-hour session window	Enforced; free-form text outside the window is rejected	Not enforced in v1
Rate limits	Real per-number throughput limits (80 msgs/sec default)	Only via X-Mock-Force-Error: 130429
Error injection header	Does not exist	X-Mock-Force-Error, mock-only
Encryption, message quality, billing	Present	Absent

None of these divergences sit on Comdove's text-message happy path or its error-handling path, which is exactly why the slice in section 2 was chosen.